

Laboratorio # 1

Base de Datos Documentales

Introducción a MongoDB

MongoDB es una base de datos de documentos, de código abierto que ofrece alto rendimiento, alta disponibilidad y escalamiento automático.

DOCUMENTOS

Un registro en MongoDB es un documento, representado por una estructura de datos compuesta por pares de campo y valor. Los documentos en MongoDB poseen formato JSON. Los valores de los campos pueden incluir otros documentos, arreglos y arreglos de documentos.

```
{  
  nombre: "Luis",  
  edad: 34,  
  estatus: "A",  
  grupos: ["noticias", "deportes"]  
}
```

← Campo: valor
← Campo: valor
← Campo: valor
← Campo: valor

COMANDOS BÁSICOS

En el cliente mongo, los comandos son ejecutados a través del objeto db. Por ejemplo:

- `db.help()` à Proporciona una lista de los comandos que se pueden ejecutar
- `db.stats()` à Proporciona estadísticas sobre la base de datos



Probá los comandos mediante el cliente Shell para ejecutar consultas: mongo (Previamente debe iniciar el proceso para el servidor mongod)

NOTA:

Al tratarse de una consola basada en JavaScript, si ejecuta un comando y omite los paréntesis (), se mostrará el cuerpo del método en lugar de la ejecución del mismo. Por ejemplo, si escribe `db.help` (sin paréntesis), podrá observar la implementación del método help.

Probemos el comando `db.stats()`:



```
> db.stats()
{
  "db" : "test",
  "collections" : 0,
  "objects" : 0,
  "avgObjSize" : 0,
  "dataSize" : 0,
  "storageSize" : 0,
  "numExtents" : 0,
  "indexes" : 0,
  "indexSize" : 0,
  "fileSize" : 0,
  "nsSizeMB" : 0,
  "dataFileVersion" : {
  },
  "ok" : 1
}
```

1. El método global *use* permite seleccionar la base de datos (No importa si la base de datos no existe aún):



```
> use lab
switched to db lab
```

La primera colección que crearemos de hecho generará la base de datos lab.

2. Una vez seleccionada la base de datos, es posible consultar todas las colecciones que posee:



```
> db.getCollectionNames()
[ ]
```

Al ejecutar el comando debe obtener un arreglo vacío. No es necesario crear un esquema para la colección, puede simplemente insertar un nuevo documento en la nueva colección.

3. Use el comando *insert* para insertar un nuevo documento, suministrando el documento a insertar:



```
> db.players.insertOne({name: 'Aaron Appindangoye', height: 182,
weight: 187})
```

El comando anterior ejecuta *insert* contra la colección *players*, indicando como argumento el documento a insertar.

4. Luego de insertar el documento a la colección *players*, ejecuta nuevamente el comando `db.getCollectionNames()`:



```
> db.getCollectionNames()  
[ "players"]
```

El resultado muestra dos colecciones: *players* y *system.indexes*. La colección *system.indexes* es creada una vez por la base de datos, y contiene la información correspondiente a los índices.

5. Usa el comando *find* contra *players* para retornar los documentos que posee la colección:



```
> db.players.find()  
{ "_id" : ObjectId("5b8c6be03c3b6184ee1b693f"), "name" : "Aaron  
Appindangoye", "height" : 182, "weight" : 187 }
```

Adicional a los datos que se especificaron al insertar el documento, se observa un campo `_id`. Todo documento debe tener un campo `_id` único. Se puede asignar uno, o dejar que MongoDB genere automáticamente el *ObjectId* correspondiente.

6. Por defecto el campo `_id` es indexado, lo que explica porque la colección *system.indexes* fue creada. Para consultar la colección *system.indexes*, ejecuta:



```
> db.system.indexes.find()  
{ "v" : 1, "key" : { "_id" : 1 }, "ns" : "lab.players", "name" : "_id_"  
}
```

Se obtiene como respuesta el nombre del índice, la base de datos y la colección en la que se creó y los campos incluidos.

7. Agrega un nuevo documento a la colección *players*:



```
> db.players.insertOne({name:'Edinson Cavani', height: 182, dob: new  
Date (1987,2,14,0,0)})
```

Nuevamente utiliza *find* para listar los documentos:



```

> db.players.find()
{ "_id" : ObjectId("5b8c75333c3b6184ee1b6943"), "name" : "Aaron Appindangoye", "height" : 182, "weight" : 187 }
{ "_id" : ObjectId("5b8c753b3c3b6184ee1b6944"), "name" : "Edinson Cavani", "height" : 182, "dob" : ISODate("1987-03-14T00:00:00Z") }

```

8. Elimina los documentos de la colección *players* con el comando `deleteMany` Si no indicas ningún selector, borrará todos los documentos:



```

> db.players.deleteMany({})
>

```

Si utilizas el comando *find* se mostrará que la colección ya no posee elementos:

9. Realiza los siguientes *insertOne* para tener más datos:

```

> db.players.insert({name:'Luka Modric', height: 180, weight:143, dob: new Date (1985,9,9,0,0), preferred_foot: 'right', hobbies:['Playing football','Watching TV series','Swimming']});
> db.players.insert({name:'Harry Kane', weight:143, dob: new Date (1993,7,28,0,0), preferred_foot: 'right', hobbies:['Watching Movies','Swimming']});
> db.players.insert({name:'Neymar', height: 175, weight:150, dob: new Date (1992,2,5,0,0), preferred_foot: 'right', hobbies:['Video games','Watching TV series','Swimming']});
> db.players.insert({name:'David Silva', height: 170, weight:148, dob: new Date (1986,8,1,0,0), preferred_foot: 'left', hobbies:['Playing football', 'Watching Movies']});
> db.players.insert({name:'Eden Hazard', height: 172, weight:163, dob: new Date (1991,1,7,0,0), preferred_foot: 'right', hobbies:['Watching Movies','Video games','Watching TV series']});
> db.players.insert({name:'Antoine Griezmann', height: 175, weight:148, dob: new Date (1991,3,21,0,0), preferred_foot: 'left', hobbies:['Video games','Swimming']});
> db.players.insert({name:'Lionel Messi', height: 170, weight:159, dob: new Date (1987,6,24,0,0), preferred_foot: 'left', hobbies:['Watching Movies','Video games','Watching TV series','Swimming']});
> db.players.insert({name:'Cristiano Ronaldo', height: 185, weight:176, dob: new Date (1985,2,5,0,0), preferred_foot: 'left', hobbies:['Video games','Swimming']});
> db.players.insert({name:'Toni Kroos', height: 182, weight:172, dob: new Date (1990,1,4,0,0), preferred_foot: 'right', hobbies:['Watching Movies','Video games','Watching TV series']});
> db.players.insert({name:'Sergio Ramos', height: 182, weight:165, dob: new Date (1986,3,30,0,0), preferred_foot: 'right', hobbies:['Video games','Watching TV series']});
> db.players.insert({name:'Samuel Umtiti', height: 180, weight:165, dob: new Date (1993,11,14,0,0), preferred_foot: 'left', hobbies:['Playing football','Swimming']});
> db.players.insert({name:'Paulo Dybala', height: 175, weight:161, dob: new Date (1993,11,15,0,0), preferred_foot: 'left', hobbies:['Playing football','Watching TV series','Swimming']});

```

Selectores

Los selectores permiten consultar documentos particulares dentro de la colección:

- `{field: value}` es usado para encontrar cualquier documento donde *field* sea igual a *value*
- `{field1: value1, field2: value2}` permite realizar una declaración *and*
- Los selectores especiales *\$lt*, *\$lte*, *\$gt*, *\$gte* y *\$ne* son usados por las operaciones menor que, menor o igual que, mayor que, mayor o igual que y no igual

Por ejemplo, para obtener todos los jugadores que tengan `preferred_foot` left y pesen más de 170 libras (`weight > 170`), se puede utilizar el siguiente comando:



```
> db.players.find({preferred_foot: 'left', weight:{$gt:170}})
{ "_id" : ObjectId("5b8c82ee3c3b6184ee1b6961"), "name" : "Cristiano
Ronaldo", "height" : 185, "weight" : 176, "dob" : ISODate("1985-03-
05T00:00:00Z"), "preferred_foot" : "left", "hobbies" : [ "Vi
deo games", "Swimming" ] }
```

El operador *\$exists* es usado para determinar la presencia o ausencia de un campo, por ejemplo para ver que documentos de la colección *players* no posee el campo *height*, ejecuta:



```
> db.players.find({height:{$exists:false}})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6967"), "name" : "Harry Kane",
"weight" : 143, "dob" : ISODate("1993-08-28T00:00:00Z"),
"preferred_foot" : "right", "hobbies" : [ "Watching Movies", "Swim
ming" ] }
```

Para este ejemplo se debe retornar un solo documento, el documento que no posee el campo.

En caso de que se desee utilizar el operador *OR* en lugar de *AND*, se utiliza *\$or* seguido de un arreglo con los valores que se desean evaluar:



```
> db.players.find({preferred_foot:'right', $or:[{hobbies:'Playing
football'},{hobbies:'Video games'}, {weight:{$lt:150}}]})
```

El comando anterior retorna todos los jugadores con *preferred_food* igual a *right*, que tengan como hobbies *Playing football* o *Video games* o que pesen menos de 150 libras.

EXPRESIONES REGULARES

MongoDB soporta expresiones regulares compatibles con Perl, para consultas basadas en patrones de coincidencia identificados en las cadenas de caracteres.

1. Consulta todos los jugadores cuyo nombre comience por la letra S:



```
> db.players.find({name: { $regex: "^S"}})
```

2. Consulta todos los jugadores cuyo nombre terminen con la letra o:



```
> db.players.find({name: { $regex: "oS"}})
```

COMANDOS DE ACTUALIZACIÓN

En su forma más simple, la operación *updateOne* tiene dos argumentos: el selector (*where*) a usar y el documento a actualizar.

3. Por ejemplo, el jugador que lleva por nombre Lionel Messi, ha subido de peso, por ello debemos actualizar el campo *weight*:



```
> db.players.updateOne({name: 'Lionel Messi'} weight:180})
```

Si consultamos el registro actualizado:



```
> db.players.find({name: 'Lionel Messi'})
```

Descubrirás la primera sorpresa del *update*: No se encuentran documentos porque el segundo parámetro suministrado es utilizado para reemplazar el original. En otras palabras, el *update* encontró un documento por *name* y reemplazó el documento completo con un nuevo documento (el segundo parámetro). En algunas situaciones esto es ideal y se puede aprovechar para algunas actualizaciones realmente dinámicas.

Si buscamos el documento por el campo actualizado (*weight:180*), nos mostrará el nuevo documento que se creó cuando se realizó el *update*:



```
> db.players.find({weight:180})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696c"), "weight" : 180 }
```

4. Cuando lo que se quiere es modificar uno o algunos de los campos, se recomienda utilizar el modificador `$set`. Por ejemplo:



```
> db.players.update({weight:180},{ $set: {name:'Lionel Messi', height:
170, dob: new Date (1987,6,24,0,0), preferred_foot: 'left',
hobbies:['Watching Movies','Video games','Watching TV
series','Swimming']}})
```

Esto va a reestablecer los campos perdidos. No se sobrescribe el nuevo *weight* ya que no se especificó.

Si ejecutamos:



```
> db.players.find({name: 'Lionel Messi'})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696c"), "dob" : ISODate("1987-
07-24T00:00:00Z"), "height" : 170, "hobbies"
: [ "Watching Movies", "Video games", "Watching TV series",
"Swimming" ], "name" : "Lionel Messi", "preferred_foot" : "left",
"weight" : 180 }
```

Obtenemos el resultado esperado. Por lo tanto, la forma correcta de haber actualizado el *weight* en primer lugar es:



```
> db.players.updateOne({name: 'Lionel Messi'},
```

5. Adicionalmente a `$set`, podemos aprovechar otros modificadores para hacer actualizaciones. Todos estos modificadores adicionales de `update` operan directamente en los campos, entonces el documento completo no será reemplazado o borrado. Por ejemplo, el operador `$inc` es usado para incrementar o decrementar de acuerdo al parámetro que se utilice (positivo o negativo, respectivamente).

Si a Luka Modric se le adjudicó incorrectamente un valor de 180 para el parámetro *height*. Primero, consultemos la estatura actual que posee Modric:



```
> db.players.find({name: 'Luka Modric'})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6966"), "name" : "Luka Modric",
  "height" : 180, "weight" : 143, "dob" : ISODate("1985-10-09T00:00:00Z"), "preferred_foot" : "right", "hobbies" : [
    "Playing football", "Watching TV series",
    "Swimming" ] }
```

Ahora, descuenta (-5) para corregir el error y verifica que se haya corregido:



```
> db.players.updateOne({name: 'Luka Modric'},{$inc:{height:-5}})
> db.players.find({name: 'Luka Modric'})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6966"), "dob" : ISODate("1985-10-09T00:00:00Z"), "height" : 175, "hobbies" : [ "Playing football",
  "Watching TV series", "Swimming" ], "name" : "Luka Modric",
  "preferred_foot" : "right", "weight" : 143 }
```

- Si David Silva de pronto desarrolla un gusto por la natación, podemos agregarlo a su campo hobbies con el modificador `$push`, ejecutando:



```
> db.players.updateOne({name: 'David Silva'},{$push:
  {hobbies:'Swimming'}})
```

- Los *updates* son completamente compatibles con *upserts*. Un *upsert* actualiza el documento si lo encuentra o lo inserta si no lo encuentra. *Upserts* son útiles de emplear en determinadas situaciones. Para habilitar *upserting* fijamos un tercer parámetro a `true`.

Un ejemplo de este caso es un contador de visitas a un sitio web. Si se desea mantener un recuento total, se tendría que validar si el registro ya existe para la página, y basado en eso decidir si ejecutar un *update* o un *insert*.

Con el tercer parámetro omitido (o establecido en *false*), la ejecución de la siguiente instrucción no tendrá ningún resultado:



```
> db.hits.update({page: 'players'}, {$inc:{hits:1}});
> db.hits.find()
```

Sin embargo, si se permiten *upserts* (tercer parámetro *true*), los resultados son diferentes:



```
> db.hits.update({page: 'players'}, {$inc:{hits:1}}, {upsert:true});
> db.hits.find()
{ "_id" : ObjectId("5b8c98570a2265f4e9a3ce7e"), "hits" : 1, "page" :
  "players" }
```


Dado que no existen documentos con un campo *page* igual a *players*, se inserta un nuevo documento. Si se ejecuta el comando de nuevo, el documento existente se actualizará y *hits* se incrementa a 2:



```
> db.hits.updateOne({page: 'players'}, {$inc:{hits:1}},
  {upsert:true});
> db.hits.find()
"players" }
```

- Hasta el momento, en los ejemplos anteriores se ha actualizado un único documento. Sin embargo, si se ejecuta el siguiente comando:



```
> db.players.updateMany({}, {$set:{active:true}});
> db.players.find({active:true})
```

Obtendremos todos los jugadores (active: true).

SELECCIÓN DE CAMPOS

El comando *find* posee un segundo parámetro opcional. Este parámetro es la lista de los campos que se desean obtener.

- Se puede obtener todos los nombres de *players* ejecutando:



```
> db.players.find(null, {name:1});
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6967"), "name" : "Harry Kane" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6968"), "name" : "Neymar" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696a"), "name" : "Eden Hazard" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696c"), "name" : "Lionel Messi" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696e"), "name" : "Toni Kroos" }
{ "_id" : ObjectId("5b8c869f3c3b6184ee1b6971"), "name" : "Paulo Dybala" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6966"), "name" : "Luka Modric" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696b"), "name" : "Antoine
Griezmann" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696d"), "name" : "Cristiano
Ronaldo" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696f"), "name" : "Sergio Ramos" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6970"), "name" : "Samuel Umtiti" }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b6969"), "name" : "David Silva" }
```

Por defecto, el campo `_id` siempre es retornado. Podemos excluirlo explícitamente especificando `{name:1, _id:0}`.



```
> db.players.find(null, {name:1, _id:0});
{ "name" : "Harry Kane" }
{ "name" : "Neymar" }
{ "name" : "Eden Hazard" }
{ "name" : "Lionel Messi" }
{ "name" : "Toni Kroos" }
{ "name" : "Paulo Dybala" }
{ "name" : "Luka Modric" }
{ "name" : "Antoine Griezmann" }
{ "name" : "Cristiano Ronaldo" }
{ "name" : "Sergio Ramos" }
{ "name" : "Samuel Umtiti" }
{ "name" : "David Silva" }
```

- El método `sort` permite ordenar los resultados por los campos que se indiquen. Se utiliza 1 para orden ascendente y -1 para descendente. Por ejemplo, podemos consultar los jugadores más altos primero:



```
> db.players.find().sort({height:-1})
```

Organiza los resultados por nombre y luego por peso:



```
> db.players.find().sort({name:1, weight:-1})
```

- También es posible paginar los resultados, a través de los métodos *limit* y *skip*. Para obtener el segundo y tercer jugador más pesado, ejecuta el siguiente comando:



```
> db.players.find().sort({weight:-1}).limit(2).skip(1)
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696d"), "active" : true, "dob" :
ISODate("1985-03-05T00:00:00Z"), "height"
: 185, "hobbies" : [ "Video games", "Swimming" ], "name" : "Cristiano
Ronaldo", "preferred_foot" : "left", "weigh
t" : 176 }
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696e"), "active" : true, "dob" :
ISODate("1990-02-04T00:00:00Z"), "height"
: 182, "hobbies" : [ "Watching Movies", "Video games", "Watching TV
series" ], "name" : "Toni Kroos", "preferred_foot" : "right", "weight" :
172 }
```

NOTA:

Usando *limit* en conjunto con *sort*, es una buena forma de evitar encontrarnos con problemas cuando hacemos ordenamiento en campo sin índice.

- Se puede ejecutar *count* directamente en una colección, para conocer el número de documentos que cumplen con determinada condición:



```
> db.players.countDocument({hobbies: 'Swimming'})
9
```

Otra forma de ejecutar el mismo resultado es:



```
> db.players.find({hobbies: 'Swimming'}).countDocument()
```

DOCUMENTOS EMBEBIDOS

En MongoDB el valor de un campo puede ser otro documento JSON, esto es útil por ejemplo para simular JOINS.

- Incorporemos al jugador Cristiano Ronaldo, un documento con información correspondiente a su equipo:



```
> db.players.updateOne({name: 'Cristiano Ronaldo'}, {$set: {team: {
team_long_name: 'Juventus', team_short_name: 'JUV'}}})
```

- Los documentos embebidos también pueden ser accedidos usando la notación de punto:



```
> db.players.find({'team.team_short_name': 'JUV'})
{ "_id" : ObjectId("5b8c869c3c3b6184ee1b696d"), "active" : true, "dob" :
ISODate("1985-03-05T00:00:00Z"), "height"
: 185, "hobbies" : [ "Video games", "Swimming" ], "name" : "Cristiano
Ronaldo", "preferred_foot" : "left", "team"
: { "team_long_name" : "Juventus", "team_short_name" : "JUV" }, "weight"
: 176 }
```

INDICES Y ADMINISTRACIÓN

Los índices en MongoDB funcionan muy parecido a los índices en una base de datos relacional: ayudan a mejorar el rendimiento de los queries y ordenados.

1. Los índices son creados con el comando *createIndex*:



```
> db.players.createIndex({name:1})
```

2. Los índices pueden ser borrados usando *dropIndex*:



```
> db.players.dropIndex({name:1})  
{ "nIndexesWas" : 2, "ok" : 1 }
```

3. Un índice único se crea utilizando el parámetro *unique*:



```
> db.players.createIndex({name:1},{unique:true})
```

4. Los índices pueden ser creados en campos embebidos (usando la notación de punto) y en campos de arreglos. También, es posible crear índices compuestos:



```
> db.players.createIndex({name:1, weight:1})
```

5. Para validar si los *queries* están usando índices o no, se puede utilizar el método *explain*:



```
> db.players.find().explain("executionStats")  
{  
  "cursor" : "BasicCursor",  
  "isMultiKey" : false,  
  "n" : 12,  
  "nscannedObjects" : 12,  
  "nscanned" : 12,  
  "nscannedObjectsAllPlans" : 12,  
  "nscannedAllPlans" : 12,  
  "scanAndOrder" : false,  
  "indexOnly" : false,  
  "nYields" : 0,  
  "nChunkSkips" : 0,  
  "millis" : 0,  
  "indexBounds" : {  
  },  
  "server" : "labmongo:27017"  
}
```

La salida indica que un `BasicCursor` fue usado (lo que implica no indexado), y 12 objetos fueron escaneados.

6. Si se modifica el *querie* para que use un índice, veremos que *BtreeCursor* fue usado, así como el índice utilizado para realizar el *request*:



```
> db.players.find({name: 'Lionel Messi'}).explain("executionStats")
{
  "cursor" : "BtreeCursor name_1",
  "isMultiKey" : false,
  "n" : 1,
  "nscannedObjects" : 1,
  "nscanned" : 1,
  "nscannedObjectsAllPlans" : 1,
  "nscannedAllPlans" : 1,
  "scanAndOrder" : false,
  "indexOnly" : false,
  "nYields" : 0,
  "nChunkSkips" : 0,
  "millis" : 0,
  "indexBounds" : {
    "name" : [
      [
        "Lionel Messi",
        "Lionel Messi"
      ]
    ]
  },
  "server" : "labmongo:27017"
}
```

PARTE PRÁCTICA

1. Construye en MongoDB la colección o las colecciones necesarias, para representar los siguientes datos, correspondientes a las Bandas de músicos jóvenes de la Ciudad de Buenos Aires:

NOMBRE DEL SOLISTA	GENERO	ESTILO	FECHA_INSCRIPCION	DISCOS	BARRIO	INTEGRANTES
VER K BITCH	INSTRUMENTAL		20/01/2018	DIOS AGUJERO NEGRO (2008); APICHONADOS (2007); MERIDIANO (2006); CAUSALIDADES (2005); PLANETA ESMERALDA (2001)	VERSALLES	1
TROTAMUNDOS	INDIE		30/11/2017	HECHO BOLITA (2014)	VILLA LURO	4
EFFECTO ALFONS	ROCK	POWER TRIO	27/11/2017	EFFECTO ALFONS (2000)	BARRACAS	3
MARCELO GIULLITTI	SOLISTA		26/11/2017		AGRONOMIA	1
AFTERLIFE	ROCK	ROCK ALTERNATIVO	14/11/2017		BALVANERA	5
VIRGINIA FERREYRA	ROCK	ROCK POP	14/11/2017		VILLA DEL PARQUE	1
LMV	POP		10/11/2017		LA LUCILA	6
EFFECTO ALFONS	ROCK	POWER TRIO	06/11/2017	EFFECTO ALFONS (1995)	BARRACAS	3
TANTAS PREGUNTAS	PUNK	PUNK ROCK	27/10/2017	DESPUES DE TODO (2006); LIBRE ALBEDRIO (2006)	MORENO	3
TAL VEZ DE PASO	POP	POP ROCK	26/10/2017		PUERTO MADERO	4
LA SURTIDA FOLCK	FOLKLORE		25/10/2017		BERAZATEGUI	7
JAYDEE M	HIP HOP / RAP		24/10/2017		BARRACAS	1

Posteriormente, ejecuta los comandos que permitan obtener los siguientes resultados:

- Todas las bandas en orden creciente de acuerdo al número de integrantes
- Las dos bandas con mayor número de integrantes
- Suma un integrante más a todas las bandas
- Bandas cuyo género es Rock
- Bandas que han lanzado disco en el año 2006
- Cantidad de Bandas en el barrio Barracas

Docentes: Joselyn C. Camero • Nelson Ponzoni
Septiembre 2018